

Test Assessment — AI Engineer (Computer Vision)

Format: take-home, up to 1 week calendar time.

Compute: use your own computer or a free Colab/Kaggle GPU. **If training is blocked by computational limitations or another issue, email brunoscholles@antonigrobotics.com as soon as possible with what failed, what was tried and what would be needed.**

Delivery: Git repository + a PDF report (4~6 pages). Reproducibility matters: pinned dependencies, a script or notebook that reruns inference end to end.

Scenario

Our crop-monitoring module counts raspberries from images taken by a robot driving greenhouse rows. The fruits are small, clustered and partially occluded, and the same fruit appears in overlapping image tiles and in consecutive frames. This assessment reproduces the core of that problem on a public dataset.

Dataset

Use [MinneApple](#) (apple detection/counting, public) or any comparable public small-fruit dataset (e.g. a strawberry/berry detection set from Roboflow Universe). State your choice and its limitations. Use a modest subset if training time or dataset size is a concern. **If the full image set is too large, you do not need to train with everything; around 500 images is a good target.** Sound methodology on a small subset beats a big sloppy run.

Implementation constraints

- For image preprocessing, focus on OpenCV, NumPy and Pillow. Any tiling implementation must use these libraries for image operations.
- You may still use other libraries for modelling, training and evaluation (e.g. Ultralytics, PaddlePaddle, PyTorch, TensorFlow), provided the final approach remains practical for edge inference.

Tasks

1. Baseline detector

- Train (or fine-tune) an off-the-shelf detector on the dataset. You may use any reasonable ML stack (e.g. Ultralytics/YOLO, PaddlePaddle/PaddleDetection, PyTorch, TensorFlow),

but prioritise models that could realistically run on edge devices such as an NVIDIA Jetson.

- Report standard metrics on a held-out split — and explain which metric you would actually use as an acceptance threshold for a fruit-counting product, and why.

2. Small/occluded fruit: high-resolution inference strategy

- Tiled inference is recommended (overlapping crops -> per-tile detection -> merge), but you may use another strategy if you believe it is more efficient or better for your detector/dataset.
- For tiling specifically, use OpenCV, NumPy and Pillow for crop generation, coordinate transforms, stitching/merging helpers and related preprocessing.
- Handle duplicates across overlapping tiles/regions correctly.
- Compare against direct full-image inference. Show, with numbers and 2-3 example images, how your strategy improves small/distant/occluded fruit detection and where it can hurt (duplicates, borders, runtime).

3. Approximate counting

- Produce a fruit count per image, with duplicate control across tiles/regions.
- Characterise counting accuracy against ground truth (e.g. MAE / relative error over the test split) — not just detection mAP.

4. Error-mode analysis

- We don't expect perfect results. Rank your main error modes: false negatives vs false positives, occlusion, clusters, lighting, small fruit, border effects.
- For the top 2 error modes, propose what you would change in data collection or annotation (not just the model) to fix them - this mirrors how we actually work. If a problem remains unresolved, explain what you would try next if you had more time and why you believe that solution would work.

5. Deployment/inference note (short, no deployment required)

- No production deployment is required. Include a short paragraph describing what you would do with the trained model(s) to prepare for deployment/inference.

Bonus (optional, pick at most one)

- Export the model to ONNX or TensorRT and report the latency difference at a fixed input size.
- Naive cross-frame duplicate suppression on an image sequence (if your dataset has one).
- A defined annotation schema (label classes + rules) you would give an external labelling company for raspberries.

What we evaluate

Criterion	Weight
Methodology: clean splits, honest metrics	30%
High-resolution/tiled strategy & duplicate control implemented correctly	25%
Error-mode analysis: specific, ranked, actionable (incl. data-side fixes)	25%
Reproducibility & code quality	10%
Report clarity: a teammate could pick up where you left off	10%

Follow-up session (~1 h, on-site or call)

You present the report; we discuss how the approach transfers to raspberries on a moving robot (motion blur, repeated frames, Jetson budget) and how you would define the go/no-go validation for V1.